Лабораторна робота №2
Частина 2

Генеративні моделі (GAN).
Мета роботи

Метою цієї частини є вивчення теоретичних основ генеративно-змагальних мереж (Generative Adversarial Networks, GAN) та їх архітектурного варіанту – глибоких згорткових GAN (Deep Convolutional GAN, DCGAN). Буде реалізовано модель DCGAN для генерації синтетичних зображень дорожніх знаків на основі датасету German Traffic Sign Recognition Benchmark (GTSRB). Основна увага приділяється аналізу процесу навчання, оцінці якості згенерованих зображень та дослідженню їх потенціалу для розширення (аугментації) навчальних даних.

Теоретичний матеріал

Концепція Генеративно-Змагального Навчання

Генеративно-змагальні мережі, запропоновані Іеном Гудфеллоу та його колегами у 2014 році, є інноваційним фреймворком для навчання генеративних моделей. Основна ідея полягає в антагоністичній грі між двома нейронними мережами: генератором (Generator) та дискримінатором (Discriminator).

Цю взаємодію часто пояснюють за допомогою аналогії з фальшивомонетником та поліцією. Генератор, виступаючи в ролі "фальшивомонетника", намагається створити якомога реалістичніші підробки (наприклад, зображення), щоб обдурити поліцію. Дискримінатор, у ролі "поліції", навчається відрізняти справжні дані від підробок, створених генератором. У процесі цього змагання обидві мережі вдосконалюються: генератор вчиться створювати все більш переконливі дані, а дискримінатор стає все кращим експертом у виявленні підробок. Цей процес триває доти, доки генератор не почне створювати настільки якісні дані, що дискримінатор не зможе відрізнити їх від реальних (його точність наближається до 50%).

Формально, процес навчання описується як мінімаксна гра (minimax game), де генератор (G) намагається мінімізувати функцію, яку дискримінатор (D) намагається максимізувати.

Архітектура GA

Генератор (The Generator) - це мережа, яка перетворює випадковий вектор шуму z, взятий з певного латентного простору (зазвичай багатовимірного нормального розподілу), на синтетичний зразок даних (наприклад, зображення). Його завдання - навчитися такому перетворенню, щоб розподіл згенерованих даних був якомога ближчим до розподілу реальних даних.

Дискримінатор (The Discriminator) - це стандартний бінарний класифікатор. На вхід він отримує або реальний зразок з навчального датасету, або згенерований генератором "фейк". Його завдання - видати ймовірність того, що вхідний зразок є

реальним. Для реальних даних він повинен видавати значення, близьке до 1, а для згенерованих близьке до 0.

Ранні версії GAN, що базувалися на повнозв'язних шарах (багатошарових перцептронах), часто страждали від нестабільності навчання та низької якості згенерованих зображень. У 2015 році Алек Редфорд та інші представили DCGAN (Deep Convolutional Generative Adversarial Networks), запропонувавши набір архітектурних правил, які дозволили стабільно навчати GAN на основі згорткових мереж для генерації високоякісних зображень. Ключові архітектурні принципи DCGAN включають :

1. Заміна шарів підвибірки (pooling) на згортки з кроком (strided convolutions) у дискримінаторі (для зменшення розмірності) та на транспоновані згортки (transposed convolutions) у генераторі (для збільшення розмірності).
2. Використання пакетної нормалізації (Batch Normalization) в обох мережах (крім вихідного шару генератора та вхідного шару дискримінатора) для стабілізації навчання та запобігання "колапсу мод" (mode collapse).
3. Відмова від повнозв'язних шарів у глибоких архітектурах, що сприяє кращому вивченню просторових ознак.
4. Використання активації ReLU у всіх шарах генератора, крім вихідного, де застосовується *tanh*. У дискримінаторі використовується LeakyReLU, що дозволяє градієнтам проходити навіть через "мертві" нейрони.

Умовні генеративно-змагальні мережі (Conditional GAN).

У класичній GAN генератор формує зображення лише на основі випадкового латентного вектора zz, тому користувач не може безпосередньо визначити, який саме об’єкт буде згенеровано. У Conditional GAN (cGAN) до генератора і дискримінатора додатково подається умовна інформація yy, наприклад мітка класу. Генератор формує зображення G(z,y)G(z,y), що відповідає заданому класу, а дискримінатор оцінює правдоподібність пари «зображення–мітка». Такий підхід дає змогу керувати процесом генерації та отримувати синтетичні зображення конкретних класів.

Процес навчання GAN є ітеративним і почерговим. На кожному кроці виконуються два оновлення.

Крок 1: Навчання Дискримінатора. Дискримінатор навчається на міні-батчі, що складається з реальних зображень з датасету та згенерованих зображень. Його функція втрат є сумою двох компонентів: втрат на реальних зображеннях (де він повинен передбачати 1) та втрат на згенерованих (де він повинен передбачати 0). Зазвичай для цього використовується бінарна перехресна ентропія.

Крок 2: Навчання Генератора. Генератор навчається "обманювати" дискримінатор. Для цього генерується новий батч "фейкових" зображень, які подаються на вхід дискримінатору. Ваги дискримінатора на цьому етапі "заморожені". Функція втрат генератора розраховується на основі виходу дискримінатора, але цього разу цільовою міткою є 1. Тобто, генератор оновлює свої ваги таким чином, щоб дискримінатор з більшою ймовірністю класифікував його витвори як реальні.

Особливості навчання GAN.

Навчання GAN може бути нестабільним через необхідність одночасного балансування генератора і дискримінатора. Якщо дискримінатор навчається значно швидше, генератор може отримувати недостатньо інформативні градієнти. Однією з характерних проблем є *mode collapse* — ситуація, коли генератор створює обмежену кількість подібних зображень і не відтворює повного різноманіття навчального набору. На стабільність навчання впливають архітектура мереж, швидкість навчання, розмір

пакета, параметри оптимізатора, нормалізація та інші гіперпараметри.

Оцінювання якості генерації.

Якість GAN не можна коректно визначати лише за значеннями функцій втрат генератора і дискримінатора. Тому результати зазвичай оцінюють візуально за реалістичністю та різноманітністю зображень, а також за допомогою кількісних показників. Одним із поширених показників є Fréchet Inception Distance (FID), який оцінює відмінність між розподілами ознак реальних і згенерованих зображень. Менше значення FID зазвичай відповідає більшій подібності згенерованих даних до реальних.

Практична Реалізація: Генерація Зображень Дорожніх Знаків

Для генерації зображень дорожніх знаків ми використаємо відомий датасет German Traffic Sign Recognition Benchmark (GTSRB). Цей набір даних містить понад 50,000 зображень 43 класів дорожніх знаків Німеччини, знятих у реальних умовах з різним освітленням, ракурсами та частковими перекриттями. Для навчання GAN нам не потрібні мітки класів, оскільки це неконтрольоване навчання. Ми будемо використовувати лише зображення з навчального набору.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import kagglehub
import numpy as np
import matplotlib.pyplot as plt
import os
import cv2
import time
import pickle

# --- Перевірка GPU ---
print("TensorFlow бачить такі пристрої:")
print(tf.config.list_physical_devices())
if not tf.config.list_physical_devices('GPU'):
    print("ПОПЕРЕДЖЕННЯ: GPU не знайдено! Навчання буде на CPU.")

BUFFER_SIZE = 30000
BATCH_SIZE = 256
IMAGE_SIZE = 32
NOISE_DIM = 100
EPOCHS = 50
num_examples_to_generate = 16
NUM_CLASSES = 43

# Метод для завантаження та передобробки дани
def load_gtsrb_data(data_dir):
    train_file_path = os.path.join(data_dir, 'train.p')
    print(f"Завантаження даних з pickle-файлу: {train_file_path}")

    try:
        with open(train_file_path, 'rb') as f:
            train_data = pickle.load(f)

            if 'features' in train_data and 'labels' in train_data:
                images = train_data['features']
                labels = train_data['labels']
            else:
                print("ПОМИЛКА: Ключі 'features' або 'labels' не знайдено у файлі train.p.")
                return np.array([]), np.array([])

    except FileNotFoundError:
        print(f"ПОМИЛКА: Файл 'train.p' не знайдено за шляхом {train_file_path}")
        return np.array([]), np.array([])
    except Exception as e:
        print(f"ПОМИЛКА при читанні pickle-файлу: {e}")
        return np.array([]), np.array([])

    images = images.astype('float32')
    images = (images - 127.5) / 127.5

    labels = labels.astype('int32')

    print(f"Завантажено {len(images)} зображень та {len(labels)} міток.")
    print(f"Форма масиву зображень: {images.shape}")

    if images.shape[1] != IMAGE_SIZE or images.shape[2] != IMAGE_SIZE:
```

```
        print(f"ПОПЕРЕДЖЕННЯ: Розмір зображення {images.shape[1]}x{images.shape[2]} не збігається з
IMAGE_SIZE={IMAGE_SIZE}")

    return images, labels

print("Завантаження датасету з KaggleHub...")
path = kagglehub.dataset_download("harbhajansingh21/german-traffic-sign-dataset")
train_images, train_labels = load_gtsrb_data(path)

if train_images.size == 0:
    print("Не вдалося завантажити зображення. Вихід.")
    exit()

train_dataset = tf.data.Dataset.from_tensor_slices((train_images, train_labels))\
                               .shuffle(BUFFER_SIZE)\
                               .batch(BATCH_SIZE, drop_remainder=True)

# Модель Генератора (cGAN)
def make_generator_model(num_classes):
    # Вхід для шуму
    noise_input = layers.Input(shape=(NOISE_DIM,), name="noise_input")

    # Вхід для мітки
    label_input = layers.Input(shape=(1,), name="label_input")

    # 1. Вбудовуємо (Embed) мітку і перетворюємо її у вектор
    # Вектор розміром 50 - це гіперпараметр
    label_embedding = layers.Embedding(num_classes, 50)(label_input)
    label_embedding = layers.Flatten()(label_embedding)

    # 2. Об'єднуємо шум і вектор мітки
    merged_input = layers.Concatenate()([noise_input, label_embedding])x = layers.Dense(4*4*1024,
    use_bias=False)(merged_input) x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU()(x)
    x = layers.Reshape((4, 4, 1024))(x)

    x = layers.Conv2DTranspose(512, (5, 5), strides=(2, 2), padding='same', use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU()(x)

    x = layers.Conv2DTranspose(256, (5, 5), strides=(2, 2), padding='same', use_bias=False)(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU()(x)

    output_image = layers.Conv2DTranspose(3, (5, 5), strides=(2, 2), padding='same', use_bias=False,
activation='tanh')(x)

    # Створюємо модель з двома входами
    model = keras.Model(inputs=[noise_input, label_input], outputs=output_image)
    return model

generator = make_generator_model(NUM_CLASSES)
generator.summary()

# Модель Дискримінатора (cGAN)
def make_discriminator_model(num_classes):
    # Вхід для зображення
    image_input = layers.Input(shape=[IMAGE_SIZE, IMAGE_SIZE, 3], name="image_input")

    # Вхід для мітки
    label_input = layers.Input(shape=(1,), name="label_input")

    # 1. Вбудовуємо (Embed) мітку і перетворюємо її на "канал" зображення
    label_embedding = layers.Embedding(num_classes, IMAGE_SIZE * IMAGE_SIZE)(label_input)
    label_embedding = layers.Reshape((IMAGE_SIZE, IMAGE_SIZE, 1))(label_embedding)

    # Об'єднуємо канал мітки із зображенням
    merged_input = layers.Concatenate()([image_input, label_embedding]) # (32, 32, 4)

    x = layers.Conv2D(64, (5, 5), strides=(2, 2), padding='same')(merged_input)
    x = layers.LeakyReLU()(x)
    x = layers.Dropout(0.3)(x)

    x = layers.Conv2D(128, (5, 5), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU()(x)
    x = layers.Dropout(0.3)(x)

    x = layers.Conv2D(256, (5, 5), strides=(2, 2), padding='same')(x)
    x = layers.LeakyReLU()(x)
    x = layers.Dropout(0.3)(x)

    x = layers.Flatten()(x)
    output_logit = layers.Dense(1)(x) # Вихід - один логіт

    # Створюємо модель з двома входами
    model = keras.Model(inputs=[image_input, label_input], outputs=output_logit)
    return model

discriminator = make_discriminator_model(NUM_CLASSES)
```

```
discriminator.summary()

# Функції Втрат та Оптимізатори
cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output) * 0.9, real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)

# Чекпоінти
checkpoint_dir = './training_checkpoints'
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(generator_optimizer=generator_optimizer,
                                 discriminator_optimizer=discriminator_optimizer,
                                 generator=generator,
                                 discriminator=discriminator)

latest_ckpt = tf.train.latest_checkpoint(checkpoint_dir)
if latest_ckpt:
    print(f"Відновлення з останнього чекпоінта: {latest_ckpt}")
    status = checkpoint.restore(latest_ckpt).expect_partial()
    print("Чекпоінт успішно відновлено.")
else:
    print("Чекпоінти не знайдено. Початок навчання з нуля.")


# Крок Навчання (cGAN)
@tf.function
def train_step(data):
    images, labels = data
    labels = tf.cast(labels, 'int32')

    noise = tf.random.normal([BATCH_SIZE, NOISE_DIM])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator([noise, labels], training=True)

        real_output = discriminator([images, labels], training=True)
        fake_output = discriminator([generated_images, labels], training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))

    return gen_loss, disc_loss

output_dir = 'gan_images'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

seed_noise = tf.random.normal([num_examples_to_generate, NOISE_DIM])
seed_labels = tf.constant(np.arange(num_examples_to_generate), dtype='int32')

def generate_and_save_images(model, epoch, test_inputs):
    test_noise, test_labels = test_inputs

    predictions = model([test_noise, test_labels], training=False)

    fig = plt.figure(figsize=(4, 4))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i+1)
        plt.imshow((predictions[i, :, :, :] * 0.5) + 0.5)
        plt.title(f"Клас: {test_labels[i].numpy()}")
        plt.axis('off')

    plt.savefig(os.path.join(output_dir, f'image_at_epoch_{epoch:04d}.png'))
    # plt.show()
    plt.close(fig)

# Основний цикл навчання
def train(dataset, epochs):
    print("Generating initial image (from noise)...")
    generate_and_save_images(generator, 0, (seed_noise, seed_labels))
```

```
    for epoch in range(epochs):
        start = time.time()

        total_gen_loss = 0
        total_disc_loss = 0
        batch_count = 0

        tf.print(f"Starting epoch {epoch+1}")

        for image_batch, label_batch in dataset:
            gen_loss, disc_loss = train_step((image_batch, label_batch))
            total_gen_loss += gen_loss
            total_disc_loss += disc_loss
            batch_count += 1

        generate_and_save_images(generator, epoch + 1, (seed_noise, seed_labels))

        if (epoch + 1) % 5 == 0:
            checkpoint.save(file_prefix=checkpoint_prefix)
            print(f"Saved checkpoint for epoch {epoch + 1}")

        avg_gen_loss = total_gen_loss / batch_count
        avg_disc_loss = total_disc_loss / batch_count

        print(f'Time for epoch {epoch + 1} is {time.time()-start:.2f} sec')
        print(f'Avg Gen Loss: {avg_gen_loss}, Avg Disc Loss: {avg_disc_loss}')

    generate_and_save_images(generator, epochs, (seed_noise, seed_labels))
    print("Training finished.")

print("Starting training...")
train(train_dataset, EPOCHS)
```

Візуалізація згенерованих зображень на різних етапах навчання є найкращим способом оцінити прогрес генератора. На початкових епохах зображення виглядають як випадковий шум, але з часом модель вчиться створювати все більш структуровані та впізнавані форми, що нагадують дорожні знаки.

На зображенні видно, як з випадкового шуму (Epoch 1) поступово з'являються контури та кольори, характерні для дорожніх знаків. На пізніх епохах (Epoch 50) можна розрізнити круглі та трикутні форми, а також кольорові схеми (червоний, синій), що відповідають реальним знакам.

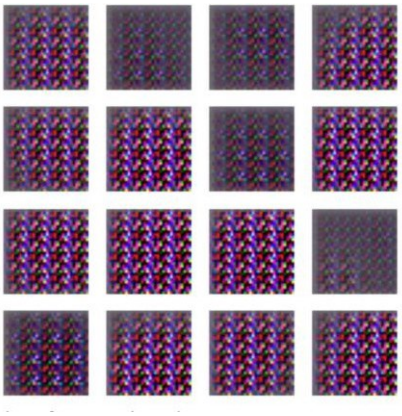


Рис. 1. Перша епоха.

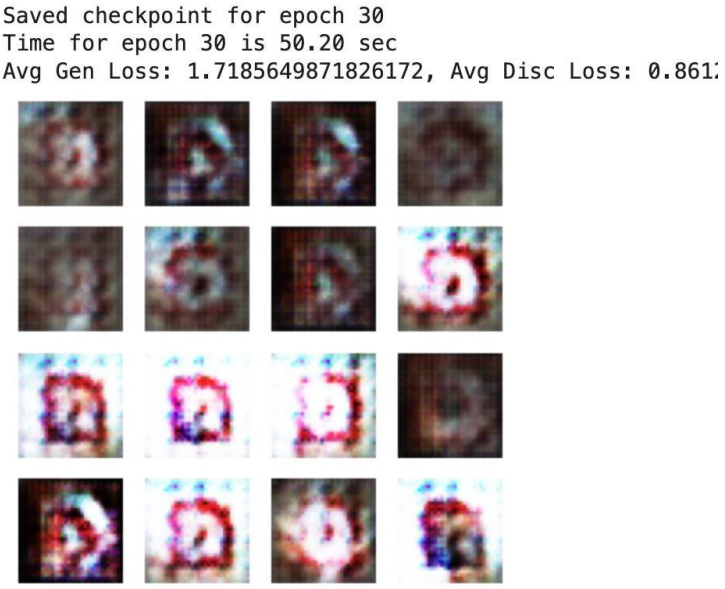


Рис. 2. Тридцята епоха.



Рис. 3. П'ятдесята епоха з пронумерованими класами об'єктів.

Фінальні згенеровані зображення демонструють, що модель DCGAN успішно вивчила загальні характеристики дорожніх знаків: форму (коло, трикутник), колір (червоний обідок, синій фон) та наявність символів у центрі. Хоча згенеровані символи не завжди є чіткими, загальна структура знаків є реалістичною.

Ці синтетичні зображення мають великий потенціал для аугментації даних. Додавання їх до навчального набору може допомогти класифікатору дорожніх знаків стати більш стійким до варіацій освітлення, невеликих спотворень та інших умов, які могли бути недостатньо представлені в оригінальному датасеті. Це особливо корисно для рідкісних класів знаків, для яких важко зібрати велику кількість реальних прикладів.

Індивідуальні завдання

Відповідно до отриманого варіанта реалізувати та дослідити GAN-модель для генерації зображень на відкритому наборі даних.

1. Завантажити та описати набір даних: кількість зображень, класи, структура даних.

2. Підготувати дані: змінити розмір зображень, нормалізувати значення пікселів до діапазону [-1, 1].
3. Побудувати та навчити базову модель DCGAN або cGAN.
4. Виконати експеримент відповідно до індивідуального варіанта та порівняти результати з базовою конфігурацією.
5. Побудувати візуалізації: графіки втрат генератора і дискримінатора, приклади згенерованих зображень на різних епохах та результати експерименту.
6. Проаналізувати результати: якість і різноманітність генерації, стабільність навчання, можливий *mode collapse*, вплив досліджуваного параметра або модифікації.
7. Сформулювати висновки та запропонувати можливі способи покращення моделі.

Рекомендовані базові налаштування

- Генератор: `Conv2DTranspose`, `BatchNormalization`, `ReLU/LeakyReLU`, вихідний шар `tanh`.
- Дискримінатор: `Conv2D`, `LeakyReLU`, `Dropout`.
- Оптимізатор: Adam, `learning_rate = 0.0002`, `beta_1 = 0.5`.
- Функція втрат: `BinaryCrossentropy`.
- Латентний простір: `NOISE_DIM = 100`.
- Навчання: `batch_size = 64–128`, `epochs = 50–100`.

Параметри можуть змінюватися відповідно до індивідуального варіанта завдання.

№	Завдання	Метод	Тип даних	Очікуваний результат	Датасет
1	**Генерація рукописних цифр**	DCGAN	Зображення цифр	Навчити DCGAN та отримати синтетичні зображення цифр 0–9. Проаналізувати зміну якості генерації за епохами	MNIST
2	**Умовна генерація дорожніх знаків заданого класу**	cGAN	Зображення + мітки класів	Генерувати дорожні знаки вибраного класу за заданою міткою та оцінити відповідність класу	GTSRB
3	**Порівняння DCGAN та cGAN**	DCGAN / cGAN	Зображення + мітки	Порівняти некеровану та умовну генерацію за якістю, різноманітністю та відповідністю класам	GTSRB
4	**Дослідження впливу розміру латентного простору**	DCGAN або cGAN	Зображення	Навчити модель для кількох значень `NOISE_DIM`, наприклад 50, 100 і 200, та порівняти якість і різноманітність генерації	GTSRB
5	**Дослідження впливу розміру пакета**	DCGAN або cGAN	Зображення	Порівняти навчання для `batch_size`, наприклад 32, 64 і 128; проаналізувати втрати та якість зображень	GTSRB
6	**Дослідження швидкості навчання**	DCGAN або cGAN	Зображення	Порівняти кілька значень `learning_rate` та визначити їх вплив на стабільність генератора і дискримінатора	GTSRB

7	**Дослідження параметра β_1 оптимізатора Adam**	DCGAN або cGAN	Зображення	Порівняти різні значення `beta_1`, наприклад 0.3, 0.5 і 0.9, за кривими втрат та якістю генерації	GTSRB
8	**Дослідження впливу Batch Normalization**	DCGAN	Зображення	Порівняти генератор із BatchNormalization і без нього; оцінити стабільність навчання та якість результатів	GTSRB
9	**Дослідження впливу Dropout у дискримінаторі**	DCGAN	Зображення	Порівняти кілька значень Dropout та визначити їх вплив на баланс між генератором і дискримінатором	GTSRB
10	**Зміна глибини генератора**	DCGAN	Зображення	Додати або вилучити один згортковий блок генератора та оцінити вплив архітектури на якість згенерованих зображень	GTSRB
11	**Зміна глибини дискримінатора**	DCGAN	Зображення	Порівняти дискримінатори різної глибини та проаналізувати вплив їх потужності на процес навчання GAN	GTSRB
12	**Дослідження функцій активації генератора**	DCGAN	Зображення	Порівняти, наприклад, ReLU та LeakyReLU у прихованих шарах генератора; оцінити процес навчання та якість генерації	GTSRB
13	**Дослідження label smoothing**	DCGAN / cGAN	Зображення	Порівняти навчання з мітками реальних даних `1.0` і згладженими мітками, наприклад `0.9`; проаналізувати стабільність GAN	GTSRB
14	**Дослідження розподілу випадкового шуму**	DCGAN	Зображення	Порівняти нормальний та рівномірний розподіли для формування латентних векторів та оцінити різноманітність результатів	GTSRB
15	**Дослідження впливу обсягу навчальної вибірки**	DCGAN / cGAN	Зображення	Навчити модель, наприклад, на 25 %, 50 % та 100 % доступних даних та визначити вплив кількості зображень на якість генерації	GTSRB
16	**Генерація даних для рідкісних класів**	cGAN	Зображення + мітки	Вибрати один або кілька класів із меншою кількістю прикладів і згенерувати додаткові	GTSRB

				зображення для балансування вибірки	
17	**Вплив геометричної аугментації на GAN**	DCGAN / cGAN	Зображення	Додати невеликі повороти, масштабування або зміщення до реальних зображень і дослідити вплив на різноманітність генерації	GTSRB
18	**Інтерполяція в латентному просторі**	DCGAN / cGAN	Зображення	Побудувати плавний перехід між двома латентними векторами та проаналізувати зміну форми, кольору й структури згенерованих об'єктів	GTSRB
19	**Оцінка якості генерації за допомогою FID**	DCGAN / cGAN	Зображення	Обчислити FID для згенерованих і реальних зображень та зіставити кількісну оцінку з візуальною	GTSRB
20	**Використання синтетичних зображень для аугментації класифікатора**	cGAN + CNN	Зображення + мітки	Навчити класифікатор без синтетичних даних і з ними та порівняти точність, precision/recall або інші показники	GTSRB

Зміст звіту

Звіт з лабораторної роботи повинен містити:

1. Титульний аркуш із зазначенням назви лабораторної роботи, номера варіанта, прізвища студента та навчальної групи.
2. Індивідуальне завдання із зазначенням набору даних, типу GAN-моделі та досліджуваного параметра або модифікації.
3. Програмний код реалізації GAN, що включає підготовку даних, побудову генератора і дискримінатора, навчання моделі та виконання індивідуального завдання.
4. Структуру генератора і дискримінатора (model.summary()) та короткий опис їх основних шарів.
5. Результати навчання, що містять:

    - графіки функцій втрат генератора та дискримінатора;
    - приклади згенерованих зображень на різних епохах;
    - фінальні результати генерації.

6. Результати виконання індивідуального завдання – порівняння досліджуваних конфігурацій, параметрів або архітектур та відповідні візуалізації.
7. Оцінювання якості генерації: візуальна оцінка якості й різноманітності зображень, аналіз стабільності навчання та можливих проявів *mode collapse*. Для відповідних варіантів – значення FID або результати використання синтетичних даних для аугментації.
8. Аналіз результатів та висновки щодо якості генерації, впливу досліджуваного параметра та можливих способів покращення GAN-моделі.